Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ПРАКТИЧЕСКАЯ РАБОТА

Дисциплина(модуль)/предмет **МДК.03.02**

Тема: **Анализ кейсов и оценка рисков качества ПО по ISO/IEC 25010**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** _____ /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 КЕЙС И НАРУШЕННЫЕ ХАРАКТЕРИСТИКИ ISO/IEC 25010.....4
2 ПЕРВОПРИЧИНА СБОЯ (ROOT CAUSE).....5
3 ПЛАН МИТИГАЦИИ РИСКОВ.....6
ЗАКЛЮЧЕНИЕ.....8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....9

ВВЕДЕНИЕ

Цель работы: научиться применять международный стандарт качества ISO/IEC 25010 [1] для анализа реального сбоя в интеллектуальной информационной системе: определить нарушенные характеристики качества, найти первопричину и предложить план снижения рисков.

Работа выполнена по варианту 3. Помимо анализа, предложенные меры проверены на небольшой C#-программе, которая имитирует нагрузку «Чёрной пятницы» и сравнивает работу системы до и после исправлений.

1 КЕЙС И НАРУШЕННЫЕ ХАРАКТЕРИСТИКИ ISO/IEC 25010

1.1 Описание кейса

Вариант 3. Сервис встроенного инференса ONNX в интернет-магазине под нагрузкой в Чёрную пятницу стал отвечать дольше 5 секунд, вызывая каскадные таймауты во всём .NET-приложении.

Модель в формате ONNX запускается прямо внутри процесса веб-приложения (например, через ONNX Runtime [2]) и строит рекомендации товаров. В обычные дни ответ приходит за доли секунды, но при резком росте числа покупателей запросы начали накапливаться, страницы магазина перестали открываться, а соседние сервисы (корзина, оплата) стали получать таймауты.

1.2 Нарушенные характеристики

Стандарт ISO/IEC 25010 делит качество на характеристики и подхарактеристики [1]. В кейсе нарушены следующие.

Таблица 1 – Соотнесение проблемы с ISO/IEC 25010

Характеристика	**Подхарактеристика**	**Как проявилась в кейсе**
Performance Efficiency (производительность)	Time behaviour (время отклика); Capacity (предельная нагрузка)	Ответ дольше 5 с вместо долей секунды; система не рассчитана на пиковую нагрузку Чёрной пятницы
Reliability (надёжность)	Availability (доступность); Fault tolerance (устойчивость к сбоям)	Сбой одного модуля рекомендаций вызвал каскад таймаутов во всём приложении, магазин фактически недоступен
Maintainability (сопровождаемость)	Analysability (анализируемость)	Нет мониторинга задержки инференса, поэтому причину замедления сложно быстро найти

Главной нарушенной характеристикой является Performance Efficiency, а Reliability пострадала как следствие: медленный модуль потянул за собой остальные.

2 ПЕРВОПРИЧИНА СБОЯ (ROOT CAUSE)

Нужно определить, чем вызван сбой: технической ошибкой в .NET API, дрейфом данных (Data Drift), дрейфом концепции (Concept Drift) или уязвимостью самой модели [3].

Таблица 2 – Проверка версий первопричины

Версия	**Подходит ли**	**Почему**
Data Drift (изменились входные данные)	Нет	Ответы модели остались корректными, медленными, а не неверными; в кейсе нет признаков изменения входных данных
Concept Drift (изменилась зависимость)	Нет	Качество рекомендаций не падало, проблема во времени ответа
Уязвимость модели	Нет	Нет атаки или некорректных ответов
Техническая ошибка архитектуры .NET API	Да	Тяжёлый синхронный инференс в общем потоке обработки запросов

Первопричина – техническая ошибка проектирования в .NET-приложении. Инференс выполняется синхронно и блокирует потоки пула [4]. Пока число запросов небольшое, потоков хватает. В Чёрную пятницу все потоки заняты моделью, новые запросы встают в очередь, время ожидания растёт, и вызывающие сервисы срабатывают по таймауту. Дополнительно в коде нет ограничения числа одновременных вызовов, таймаута для модели и запасного ответа, поэтому один тяжёлый компонент останавливает всё приложение.

3 ПЛАН МИТИГАЦИИ РИСКОВ

Таблица 3 – Инженерные меры

Мера	**Как реализовать**	**Что решает**
Не блокировать потоки запросов	Асинхронный вызов, вынос инференса в отдельный сервис или пул [5]	Исчерпание потоков
Ограничение параллелизма (Bulkhead)	SemaphoreSlim или политика Bulkhead [6]	Перегрузка модели
Таймаут и запасной ответ	Таймаут 1 с, при превышении – список популярных товаров [7]	Ответ дольше 5 с, каскадные таймауты
Circuit Breaker	При серии сбоев временно не вызывать модель [8]	Каскадный отказ
Мониторинг задержки	Декоратор с замером времени, метрики p95, оповещение	Быстрое обнаружение проблемы
Нагрузочное тестирование и масштабирование	Тест пиковой нагрузки перед распродажей, добавление экземпляров сервиса	Заранее известная ёмкость

Меры оформлены как декораторы [9] поверх интерфейса IRecommender: каждый добавляет одну функцию (ограничение параллелизма, таймаут с запасным ответом, мониторинг), и их можно подключать независимо.

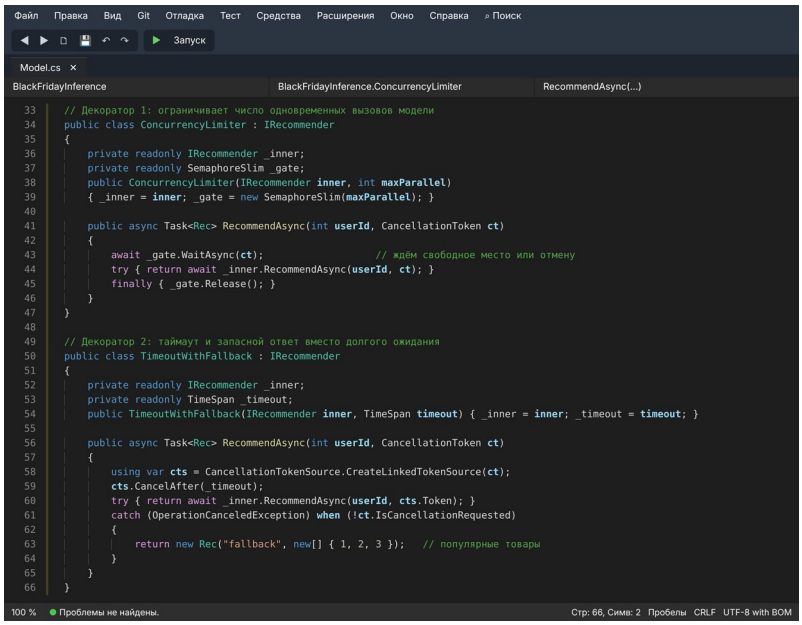


Рисунок 1 – Декораторы ConcurrencyLimiter и TimeoutWithFallback

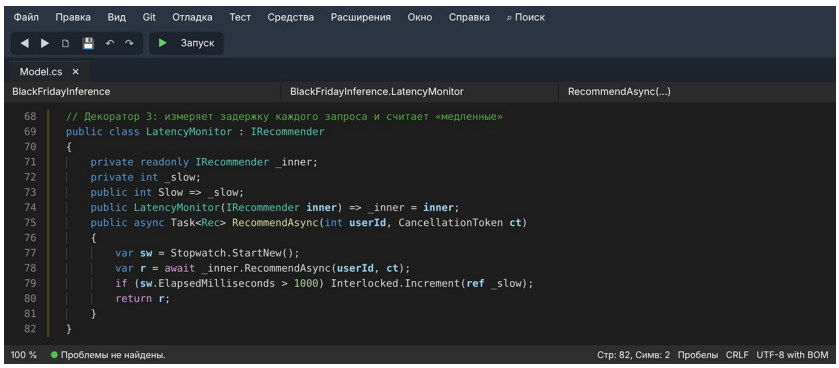


Рисунок 2 – Декоратор LatencyMonitor для замера задержки

Проверка на имитации нагрузки (200 одновременных запросов, инференс около 100 мс). В исходном варианте p95 задержки составил около 4,8 с, 4 запроса получили ответ дольше 5 с, средний ответ – 2,6 с. После митигации максимальная задержка не превысила 1,0 с, ответов дольше 5 с нет: 91 запрос обработан моделью, 109 получили запасной ответ. Мониторинг зафиксировал 110 медленных запросов, то есть проблема стала видна.

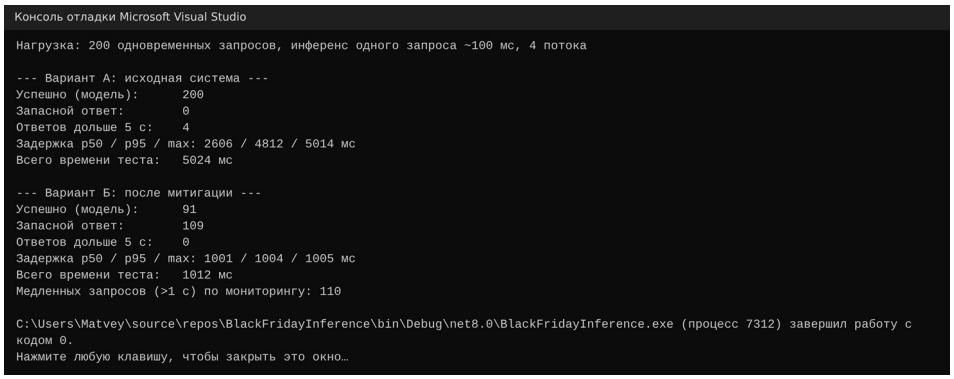


Рисунок 3 – Результат сравнения исходной и исправленной системы

Имитация использует заглушку модели вместо настоящего ONNX-файла, поэтому абсолютные цифры условны; показательно сравнение подходов.

ЗАКЛЮЧЕНИЕ

В кейсе нарушены Performance Efficiency (время отклика и ёмкость) и Reliability (доступность и устойчивость к сбоям), а Maintainability пострадала из-за отсутствия мониторинга. Первопричина – техническая ошибка архитектуры .NET-приложения: синхронный инференс без ограничений, таймаутов и запасного ответа. Дрейф данных и концепции не подходят, так как качество ответов не менялось.

Предложены асинхронный вызов, ограничение параллелизма, таймаут с запасным ответом, Circuit Breaker, мониторинг задержки и нагрузочное тестирование. На имитации нагрузки они сократили максимальный ответ с 5 с до 1 с.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ISO/IEC 25010:2011. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models [Электронный ресурс]. – URL: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010 (дата обращения: 30.10.2026).

2. Microsoft Learn. ONNX Runtime and ML.NET: Deploy ONNX models in .NET [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/machine-learning/tutorials/object-detection-onnx (дата обращения: 30.10.2026).

3. Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning [Электронный ресурс]. – URL: https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning (дата обращения: 30.10.2026).

4. Microsoft Learn. The managed thread pool [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/standard/threading/the-managed-thread-pool (дата обращения: 30.10.2026).

5. Microsoft Learn. Asynchronous programming scenarios [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/async-scenarios (дата обращения: 30.10.2026).

6. Microsoft Learn. Bulkhead pattern [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead (дата обращения: 30.10.2026).

7. Polly Documentation. Resilience strategies: timeout, circuit breaker, bulkhead [Электронный ресурс]. – URL: https://www.pollydocs.org/strategies/index.html (дата обращения: 30.10.2026).

8. Microsoft Learn. Circuit Breaker pattern [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker (дата обращения: 30.10.2026).

9. Microsoft Learn. Dependency injection in .NET: decorator and service lifetimes [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection (дата обращения: 30.10.2026).